

No. of Functional Features Included in the Solution

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Functional Features Overview:

All functional features included in the project solution, with purpose, status, and contribution. Updated to include the authentication, history and error-handling features added since the original submission.

S. N O	Feature Name	Feature Description	Module / Component	Status	Marks Contribution
1	Home Page	Introduces the project with a hero banner and background on flood forecasting	templates/home.html	Done	1
2	User Registration	Lets a visitor create an account with a username, email and password (hashed before storage)	templates/register.html, models.py (User), forms.py (RegistrationForm)	Done	1
3	User Login	Authenticates returning users by email/password, or via “Continue with Google” (Authlib OAuth)	templates/login.html, app.py (login, login_google, google_callback)	Done	1
4	Prediction Form	Collects all 10 weather/rainfall readings required by the model, with server-side validation	templates/predict.html, forms.py (PredictForm)	Done	1
5	Prediction Route & Model Inference	Validates input, scales it and classifies flood risk using the trained model; requires login	app.py (POST /predict), floods.save, transform.save	Done	1
6	Result Page + Risk Gauge	Displays an animated SVG gauge reflecting the prediction confidence/risk level	chance.html, no_chance.html, main.js (initGauge)	Done	1
7	Prediction History	Saves every prediction to the database and lets users review or delete past entries	templates/history.html, app.py (/history, /history/<id>/delete), models.py (Prediction)	Done	1
8	Light/Dark Theme Toggle	Lets users switch themes; choice persists via localStorage	main.css, main.js (initTheme)	Done	0.5
9	Ambient Rain/Lightning Animation	Canvas-based rain and lightning effects for atmosphere	main.js (initRain, initLightning)	Done	0.5
10	Flash Messaging	Shows contextual success/error/info banners after actions (login, prediction failures, etc.)	templates/partials/flash.html, app.py (flash() calls)	Done	0.5

S. N O	Feature Name	Feature Description	Module / Component	Status	Marks Co ntributio n
11	Custom Error Pages	Friendly 404 and 500 pages instead of default Flask error output	templates/errors/404.html, templates/errors/500.html	Done	0.5
12	Deployment Config	Procfile + Gunicorn setup for hosting on Render	Procfile, requirements.txt, runtime.txt	Done	1

Feature Summary

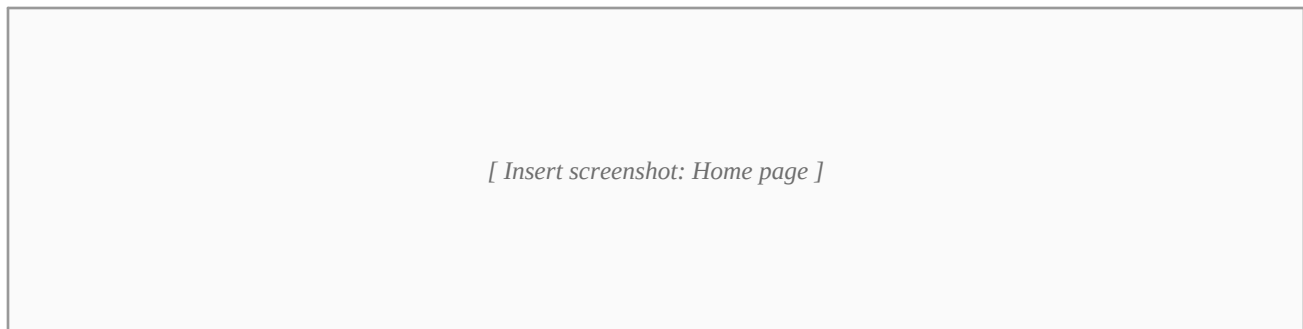
Metric	Count / Value
Total Features Planned	12
Total Features Implemented	12
Core / Must-Have Features	7 (Home, Registration, Login, Prediction Form, Prediction Route & Inference, Result Page, History)
Additional / Nice-to-Have Features	5 (Theme toggle, Ambient animation, Flash messaging, Custom error pages, Deployment config)
Features Tested & Verified	12 (manually via browser, curl, and Postman)

Feature Category Breakdown

S.NO	Category	Features in Category	Example Features
1	User Interface (UI)	4	Home page, Prediction form, Result pages, Theme toggle
2	Authentication & Accounts	2	User registration, Login (email/password + Google OAuth)
3	Backend / Data	3	Prediction route, Prediction history (view/delete), Flash messaging
4	Machine Learning	1	Scaling + prediction via Joblib model
5	Error Handling	1	Custom 404/500 error pages
6	Deployment	1	Procfile + Gunicorn on Render

Supporting Screenshots

Space reserved below for one screenshot per major feature area. Insert images at matching size; captions are pre-filled for convenience.



Home page

[Insert screenshot: Registration / Login page]

Registration / Login page

[Insert screenshot: Prediction form (10 inputs)]

Prediction form (10 inputs)

[Insert screenshot: Result page with risk gauge]

Result page with risk gauge

[Insert screenshot: Prediction history page]

Prediction history page

[Insert screenshot: Light/dark theme toggle]

Light/dark theme toggle